



CAMPUS VOICE

Project Documentation

Campus issue reporting, tracking & resolution platform

Prepared for: Development team & project stakeholders

Document scope: Problem statement, solution, approach, tech stack, workflow, features

Version: 1.0

Table of Contents

1. Problem Statement	4
Background & Context	4
Who Is Affected, and How	4
Why Existing Approaches Fall Short	4
2. Proposed Solution	4
Overview	4
Core Value Proposition	4
How It Addresses Each Pain Point	5
Expected Outcomes	5
3. Approach	6
Methodology	6
Development Phases	6
Risk Mitigation	6
Iterative Practices	7
4. Tech Stack	8
Frontend	8
Backend & Data	9
Infrastructure, Tooling & Third-Party Services	9
5. Workflow	9
Primary User Journey — Student	9
Primary User Journey — Staff	10
Primary User Journey — Admin	10
Complaint State Machine	10
Data Flow	11
Deployment Pipeline	11

Table of Contents

Team Collaboration Protocol	11
-----------------------------	----

6. Features	12
MVP — Shipped	12
Future Releases — Recommended, Not Yet Built	13

1. Problem Statement

Background & Context

College campuses generate a constant stream of physical-plant and service issues — broken Wi-Fi access points, hostel maintenance requests, faulty lab equipment, unsafe walkways, cafeteria complaints — but most institutions still route these through channels that were never designed for tracking at scale: a paper log at a front desk, a shared email inbox nobody owns, a WhatsApp group that scrolls past unresolved reports within a day, or word-of-mouth to whichever staff member happens to be nearby.

Who Is Affected, and How

- **Students** have no reliable way to report an issue and then actually follow its progress. A report that disappears into an inbox provides no confirmation it was received, no visibility into whether anyone is working on it, and no notification when it's resolved — so the same issue often gets reported multiple times by different students, or silently ignored after one attempt.
- **Facilities and department staff** lack a queue. Without a single source of truth for "what's outstanding in my department right now," prioritization becomes guesswork, and there's no record of what was fixed, when, or by whom — which makes it impossible to spot recurring problems at a given location.
- **Campus administration** has no aggregate visibility. There's no way to see which departments are overloaded, which locations generate repeat complaints, or how long issues typically take to resolve — data that would directly inform budget and staffing decisions.

Why Existing Approaches Fall Short

Generic help-desk and ticketing software (Zendesk, Freshdesk, and similar) is built for customer support, not for a distributed campus with departments organized by physical trade area (electrical, IT, civil, hostel maintenance) and a reporter base that skews toward mobile-first, infrequent use. These tools are also priced and configured for businesses, not student organizations — licensing costs and administrative overhead are disproportionate to what a single campus needs. Meanwhile, ad hoc channels (email, chat groups, paper) provide no persistent record, no status visibility, and no way to see patterns across reports over time.

2. Proposed Solution

Overview

Campus Voice is a purpose-built, web-based issue reporting and tracking platform for campus communities. Students report an issue once, with a photo and a pin on a campus map; the right department sees it immediately; staff resolve it and leave a note; the student is notified automatically. Every report, resolution, and pattern is visible to administrators in real time.

Core Value Proposition

- **One report, full visibility.** A student never has to wonder whether their report was received or ask twice — status is live and visible at every stage.
- **Automatic, immediate routing.** Reports reach the correct department's queue the moment they're filed — no manual triage bottleneck (see Workflow for the full state machine).
- **Geographic and temporal pattern visibility.** A heat map view surfaces where problems cluster and when, turning individual complaints into actionable facilities data.
- **Zero licensing cost at student-project scale.** Built entirely on Firebase's free tier plus free/low-cost third-party services (Cloudinary, OpenStreetMap), making it realistic for a single campus or department to run indefinitely.

How It Addresses Each Pain Point

Pain point	How Campus Voice addresses it
No confirmation a report was received	Report is written to a live database instantly; the student's "My Issues" page reflects it immediately, with a status stamp.
No visibility into progress	Status (Pending / Resolved) is live for every viewer via real-time listeners — no page refresh needed to see a change.
Duplicate reports for the same issue	Public campus-wide feed with upvoting lets students find and back an existing report instead of filing a new one (see Features: Future Releases for automatic duplicate detection).
Staff have no queue	Department-scoped "Pending Issues" queue, filtered automatically by the staff member's own department.
No resolution record	Resolving a ticket requires a resolution note, timestamped and retained until the configurable retention window expires.
No administrative pattern visibility	Analytics dashboard (charts, CSV export) and a geographic heat map, both filterable by date range.

Expected Outcomes

- Faster average time-to-resolution, driven by immediate routing instead of manual triage delay.
- Fewer duplicate reports once community upvoting and visibility reduce the incentive to re-report.
- Data-informed facilities planning — recurring-location and peak-time patterns become visible instead of anecdotal.
- Higher reporting rates overall, since a visible feedback loop (status changes, resolution notifications) makes reporting feel worthwhile.

3. Approach

Methodology

Campus Voice was built iteratively rather than against a single upfront spec — each phase shipped a working, testable increment, informed by direct use and bug reports from the previous phase. This reflects a pragmatic agile approach suited to a small team (or solo developer) building against real, evolving requirements rather than a fixed contract.

Development Phases

Phase	Focus	Key outcomes
	Core auth & data model	Firebase Auth + Firestore integration, role-based access (student/staff/admin), complaint CRUD, department-based routing.
2. Stabilization	Bug remediation & architecture cleanup	Fixed session-race conditions, unified routing (React Router nested routes replacing ad hoc state-based paging), centralized AuthContext, Firestore security rules written from scratch.
	Design system & UX	Token-based design system, WCAG AA contrast audit and remediation, responsive layout pass, modal-based detail views.
4. Feature expansion	Map, analytics, notifications	Leaflet-based heat map with clustering and time filters, recharts analytics dashboard with CSV export, in-app + email notification pipeline.
	Reliability & observability	Email delivery retry/logging/alerting, timeout and recovery handling for auth edge cases, scheduled data-retention cleanup.
6. Rebrand	Visual identity finalization	Full palette overhaul to the current crimson/navy/gold identity, photo-forward card redesign, form validation hardening.

Risk Mitigation

- **Single point of failure in email delivery:** mitigated with automatic retry (exponential backoff), per-attempt delivery logging, and admin alerting past a configurable failure threshold — a silent failure becomes a visible one.
- **Data growth (unbounded complaint history):** mitigated with a scheduled cleanup function that removes resolved tickets past a configurable retention window, keeping the working dataset bounded without manual intervention.

- **Firestore security misconfiguration:** mitigated by writing and version-controlling explicit security rules (rather than relying on console-configured or default rules), with role- and field-scoped permissions reviewed as part of the codebase.
- **Accessibility regressions during visual redesign:** mitigated by computing WCAG 2.1 contrast ratios programmatically for every foreground/background pairing before shipping a palette change, rather than checking by eye.

Iterative Practices

Each round of changes followed the same loop: audit the existing code to find the real root cause of a reported problem (rather than assuming), implement the fix with an inline explanation of what was wrong and why, verify with a production build and lint pass, then ship. Bugs were tracked back to root cause even when that meant revising earlier decisions — for example, a Firestore security rule was corrected mid-project after it was found to silently block community voting for anyone but a ticket's original author.

4. Tech Stack

Frontend

Technology	Role	Why this choice
React 19	UI library	Component model fits the app's role-scoped dashboards well; React 19's improvements to concurrent rendering and Suspense support the app's lazy-loaded routes (map, analytics).
Vite 7	Build tool / dev server	Fast HMR for iterative development; native ES modules keep cold-start and rebuild times low compared to older bundlers.
React Router 7	Client-side routing	Nested routes map cleanly onto the role-based layout (shared shell, per-role sub-routes); replaced an earlier ad hoc component-state paging system that broke browser back/forward and deep-linking.
Tailwind CSS v4	Styling	CSS-variable-based theme tokens (via @theme) mean the entire brand palette lives in one file — the full crimson/navy/gold rebrand touched a single CSS file for ~90% of the visual surface.
Leaflet + react-leaflet	Interactive campus map	Mature, dependency-light mapping library; free OpenStreetMap/Esri tile layers avoid a paid maps API for a student-project budget.
leaflet.heat / leaflet.markercluster	Heat map & clustering	Purpose-built plugins for density visualization and marker clustering rather than hand-rolling either.
Recharts 3	Analytics charts	Declarative, React-native charting API; lazy-loaded so its bundle weight only applies to the admin analytics route.
React Icons	Iconography	Single dependency covering all icon needs across the app.

Backend & Data

Technology	Role	Why this choice
Firebase Authentication	User auth & session	Handles password hashing, session persistence, and password-reset email delivery entirely — no custom auth server to build or secure.
Cloud Firestore	Primary database	Real-time listeners (onSnapshot) give every dashboard live updates with no polling; document model fits the complaint/user/notification shape well.
Firestore Security Rules	Authorization	Role- and field-scoped access control enforced server-side, independent of client code — a compromised or buggy client can't bypass permissions.
Cloud Functions (2nd gen)	Server-side logic	Firestore-triggered resolution notifications, a scheduled daily data-retention cleanup job, and Secret-Manager-backed credential handling for outbound email.
Cloudinary	Image hosting	Unsigned upload presets allow direct browser-to-CDN upload with no backend upload endpoint to build or pay to run.
Nodemailer + Gmail SMTP	Transactional email	No-cost email sending suitable for a student project's volume, with delivery logging and retry built around it in Cloud Functions.

Infrastructure, Tooling & Third-Party Services

Category	Choice
Hosting	Firebase Hosting (static frontend) + Cloud Functions (serverless backend logic)
Map tiles	OpenStreetMap (street) and Esri World Imagery (satellite) — both free, no API key required
Linting	ESLint, with React Hooks and Fast Refresh plugins
Package management	npm
Version control	Git

5. Workflow

Primary User Journey — Student

- Register or log in (role automatically resolves to the correct dashboard).

- Report an issue: title, description, department, priority, a pin dropped on an embedded map (or "use my current location"), and an optional photo.
- Track it from "My Issues" — status updates live, no refresh needed.
- Optionally discuss or upvote any complaint on the public campus board, not just their own.
- Receive an in-app notification (and email/push, if configured) the moment their ticket is marked resolved.

Primary User Journey — Staff

- Log in — lands on a department-scoped overview showing pending/resolved counts for their own department only.
- Work the "Pending Issues" queue, filtered automatically to their department.
- Mark a ticket resolved, required to include a resolution note.
- View the department's tickets on the map (heat map or clustered pins), filterable by date/day/hour.

Primary User Journey — Admin

- Full-campus analytics dashboard: ticket volume by department, daily trend line, date-range filtering, CSV export.
- Manage Complaints: full visibility and control over every ticket regardless of department.
- Manage Users: create accounts, change roles, remove accounts.
- Campus-wide map (not department-scoped).
- Email delivery log: per-attempt send status, error codes, and alerts when failures exceed a threshold.

Complaint State Machine

A complaint has exactly two states: **Pending** (on submission) and **Completed** (after a staff/admin resolution with a required note). Routing to a department queue is automatic and immediate — the reporting student selects the department at submission time, and staff whose profile department matches see it in their queue the instant it's written to the database, with no manual admin approval step in between. Voting and discussion are available on a complaint in either state and don't affect its status.

Data Flow

Client writes (new complaint, vote, comment, status change) go directly to Firestore, gated by security rules evaluated server-side. Every dashboard subscribes to the relevant collection via **onSnapshot**, so writes from any client propagate to every other open client in real time with no polling or manual refresh. A Firestore **onUpdate** trigger watches for a status transition to Completed and fans out to three channels in parallel: an in-app notification document, a transactional email (via Cloud Functions + Nodemailer, with retry and logging), and a push notification via FCM if the recipient's device token is on file. A separate **scheduled** function runs daily, independent of any user action, to delete resolved tickets past the retention window.

Deployment Pipeline

- Frontend: **npm run build** (Vite) produces a static bundle, deployed via **firebase deploy --only hosting**.
- Backend logic: **firebase deploy --only functions** deploys Cloud Functions; credentials are provisioned separately via **firebase functions:secrets:set** (Secret Manager), never committed to source control.
- Data layer: **firebase deploy --only firestore:rules,firestore:indexes** deploys security rules and composite indexes — both version-controlled alongside the application code rather than configured only in the console.

Team Collaboration Protocol

Environment-specific secrets (Firebase config, Cloudinary credentials, Gmail credentials) are kept out of source control via **.env** (gitignored) with a committed **.env.example** documenting every required variable, so a new contributor can see exactly what needs to be provisioned without ever seeing a real credential. Design decisions and non-obvious fixes are documented inline as code comments at the point of change, plus standalone reference docs (DESIGN_SYSTEM.md, COMPLAINT_ROUTING.md, EMAIL_DELIVERY.md, IMAGE_UPLOAD_GUIDE.md) for anything too broad for a single comment.

6. Features

MVP — Shipped

Feature	Description	Acceptance criteria
Role-based authentication	Student / staff / admin roles, each landing on a distinct dashboard.	A user can register, log in, and is routed to the dashboard matching their Firestore-stored role; wrong-role access to a route redirects rather than erroring.
Issue reporting	Title, description, department, priority, map pin, optional photo.	Submission requires all fields plus a map pin; a new complaint appears in the reporter's "My Issues" and the correct department's queue immediately.
Automatic department routing	Reports route to the matching department's staff with no manual step.	A staff account only ever sees complaints where department equals their own profile department.
Live status tracking	Pending/Completed state visible in real time to every relevant viewer.	A status change made by staff is reflected on the student's card without a page reload.
Community voting & discussion	Any signed-in user can upvote/downvote and comment on any complaint.	Vote and comment writes succeed for any signed-in user, not only the original reporter (verified against Firestore security rules).
Geographic heat map	Density/severity-weighted heat map plus clustered-pin view, with street/satellite base layers.	Map renders only complaints with coordinates; switching Heat map/Pins and Street/Satellite updates the view without a full reload.
Time-based map filtering	Filter map data by date range, day of week, and hour range.	Applying a filter updates the visible point set on both heat map and pin views identically.
Analytics dashboard	Department bar chart, daily trend line, date-range filter, CSV export.	Chart data matches the filtered date range; CSV export contains exactly the currently filtered rows.
Notifications	In-app, email, and push notification on ticket resolution.	Resolving a ticket always creates an in-app notification; email/push are attempted and logged regardless of success.

Feature	Description	Acceptance criteria
Email delivery observability	Per-attempt logging, retry with backoff, admin alerting past a threshold.	Every send attempt (success or failure) has a corresponding emailLogs document with error code/message when applicable.
Account settings	Editable profile (view/edit toggle), picture upload with crop, password/email change, notification preferences.	Profile fields are read-only until "Edit" is tapped; phone number rejects non-digit input and blocks save until exactly 10 digits or empty.
Scheduled data retention	Resolved tickets older than a configurable window are deleted automatically.	A resolved ticket persists until completedAt exceeds the retention window, then is removed by the next scheduled run.

Future Releases — Recommended, Not Yet Built

Feature	Value	Complexity
SLA / resolution-time tracking	Surfaces average time-to-resolve per department; flags tickets aging past a threshold.	Low
Aging-ticket escalation	Scheduled function re-notifies staff (or escalates to admin) for stale pending tickets.	Medium
Duplicate-report detection	Proximity + department match prompts "is this the same issue?" at submission, merging votes instead of fragmenting reports.	Medium-High
Installable offline-friendly reporting (PWA)	Report an issue with a GPS pin while offline; syncs once connectivity returns.	Low-Medium
Admin-gated triage workflow	Optional pre-routing review step for institutions that want to filter/re-route before a report reaches a department (full state-machine change — see COMPLAINT_ROUTING.md).	High
Location-name map search	Search-by-name centering on the map, pending a geocoding API key.	Low (blocked on API key)

Full detail on the current automatic-routing state machine, including exact Pending/Completed transitions and what an admin-gated alternative would require, is in COMPLAINT_ROUTING.md at the project root.